

001: Step-wise PPO 初步实验规划

范围

本文记录 Step-wise PPO 的第一版实验规划。目标是在 `verl-agent` 的多步 agent 任务中，把训练信号从“每个 step 使用同一个 episode-level reward/advantage”推进到“在环境 step 维度上计算 return、TD error 和 GAE”。这里的 step 指一次 agent 决策：agent 接收当前 context/observation，生成一个完整 action，例如 tool call 或文本动作。

当前 PPO 实现理解

标准 PPO 入口为 `verl/trainer/main_ppo.py`。训练流程包括：多轮环境交互采样、重新计算 old log probability、可选计算 reference log probability 和 critic value、计算 reward 与 advantage、最后更新 critic 和 actor。

在现有 GAE 路径中，critic value 和 advantage 是 token-shaped：

$$\delta_\ell = r_\ell + \gamma V_{\ell+1} - V_\ell, \quad A_\ell = \delta_\ell + \gamma \lambda A_{\ell+1}.$$

这对普通文本 RLHF 是自然的，但对 agentic action 不够严格，因为一个环境动作可能由多个 token 组成，而环境只在 action 粒度返回奖励。

当前 GRPO 实现理解

GRPO 不维护 critic。`env.rollout.n` 会为同一个初始任务采样多条 trajectory，这些 trajectory 共享同一个 uid，但各自有不同的 `traj_uid`。GRPO 使用 episode return 做 group-relative normalization：

$$A_i^{\text{GRPO}} = \frac{R_i - \mu_g}{\sigma_g + \epsilon}, \quad \mu_g = \frac{1}{|g|} \sum_{j \in g} R_j.$$

这个 scalar advantage 会广播到对应 response 的有效 token 上。GRPO 是干净的 non-step baseline，因为它不使用 step reward、step order 或 anchor observation。

当前 GiGPO 实现理解

GiGPO 仍然是 critic-free，但它显式利用 step 信息。它先沿 trajectory 反向计算 discounted step return：

$$G_t = r_t + \gamma G_{t+1},$$

然后在同一 task group 内按 anchor observation 构造 step group，并在 step group 内做相对归一化。最终 advantage 可以写成：

$$A_t^{\text{GiGPO}} = A_t^{\text{epi}} + w_{\text{step}} A_t^{\text{step-group}}.$$

因此，GiGPO 是评估 StepPPO 时最重要的 step-aware baseline。

Step-wise PPO 的严格定义

一条 trajectory 表示为

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}).$$

虽然 action $a_t = (y_{t,1}, \dots, y_{t,L_t})$ 由多个 token 构成, 但 value、return、TD error 和 GAE 都在环境 step 维度上定义。第一版采用严格 state-value:

$$V_t = V(s_t),$$

默认取 action 生成前最后一个 context token 的 hidden state 作为 value head 输入。取第一个 response token 或最后一个 response token 会更接近 action-conditioned value, 应作为 ablation 记录。

Step-wise TD 和 GAE 为

$$\delta_t = r_t + \gamma(1 - d_t)V^{\text{old}}(s_{t+1}) - V^{\text{old}}(s_t),$$

$$A_t^{\text{step}} = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}^{\text{step}}, \quad R_t^{\text{step}} = A_t^{\text{step}} + V^{\text{old}}(s_t).$$

Advantage 混合路线

第一版保留 GRPO/GiGPO 的 episode-level relative signal, 同时加入 learned step-wise critic correction:

$$A_t^{\text{final}} = \frac{\hat{A}_t^{\text{epi}} + w_{\text{step}}\hat{A}_t^{\text{step}}}{1 + w_{\text{step}}}.$$

最终 scalar advantage 会广播到 action response 的有效 token 上, 以复用现有 PPO actor loss。

实现计划

- 新增 `step_ppo` estimator, 不直接改 GRPO/GiGPO 逻辑。
- actor backbone 上挂一个 TRL-style value head, 不维护独立 critic 模型。
- rollout 数据保存 `uid`、`traj_uid`、`step_idx`、immediate step reward 和 old scalar state value。
- return、TD error、GAE 都沿 `traj_uid` 与 `step_idx` 排序后计算。
- actor loss 仍复用 token-level PPO clipped surrogate, 但 advantage 语义上是 step scalar。

实验矩阵

方法	是否使用 step 信息	是否使用 learned critic	角色
GRPO	否	否	non-step baseline
GiGPO	是	否	step-aware baseline
StepPPO-v1	是	是	learned step-aware method
StepPPO-v0	是	是	normalization ablation

核心指标

必须记录 validation task score、success rate、text score、valid action ratio、response length、response clipping ratio、KL、policy loss、gradient norm、value loss、value prediction statistics、timing components、GPU memory、CPU memory 和 throughput。

主要风险

主要风险包括：value target 高方差、value loss 干扰 actor backbone、token-level PPO ratio 带来的长度偏置、batch shuffle 后 step order 错乱、以及 rollout engine 同步时误包含 value-head 参数。